Final Exam: FSCT 8561 - Advanced Security Applications

- **Format:** Take-Home, Open Book
- **Instructor:** Dr. Maryam R. Aliabadi
- **Start Date:** Apr 15th at 9am
- **End Date:** Apr19th at 11:59pm

Exam Overview

You are a cybersecurity investigator who has intercepted a digital image (**evidence.png**) believed to contain a hidden manifesto from an entity known as **"The Architect."**

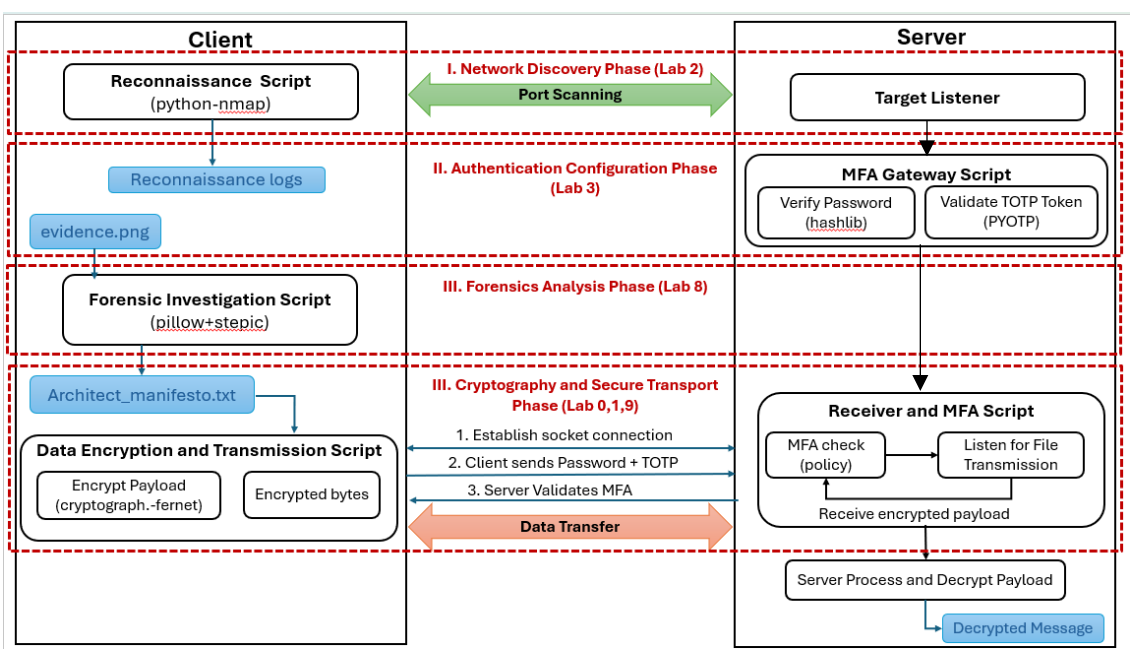
Your objective is to build a secure, end-to-end Python-based pipeline that:

- Reconnoiters the network environment
- Extracts the hidden message using steganography tools
- Protects the message with high-grade encryption
- Transmits the evidence to a remote server gated by Multi-Factor Authentication

---

System Architecture Overview

To help you visualize the end-to-end pipeline, refer to the following logic flow. Your task is to connect the individual modules you built in previous labs into this single, unified system.



Client Side:

Scan Target → Extract Message → Encrypt Message → Authenticate to Server → Send.

Server Side:

Listen for Connection → Verify 2FA → Receive File → Decrypt & Store.

Daily Milestones & Technical Requirements

Day 1: Network Reconnaissance (Lab 2)

Task: Use the python-nmap library to scan your Server environment (Local loopback or a designated target VM).

Requirement: Identify if the ports you intend to use for your Socket communication are open and identify the service versions. Save results to a log file.

System Integration Note:
Before the agent sends data, they must know if the server is reachable and which ports are open. This is your "Pre-flight Check." You are confirming that your destination (the Server) is reachable and correctly configured before you attempt to send sensitive data later in the week.

Day 2: Multi-Factor Authentication (Lab 3)

Task: Create a server-side authentication gateway.

Requirement: Implement a two-step authentication logic that your server will use to verify any incoming client connection:

- A hashed and salted password (using hashlib)
- A valid Time-based One-Time Password (using PyOTP)

Note (Security Rationale):
This task evaluates correct implementation of **multi-factor authentication principles** used in modern secure systems. You are expected to implement functional authentication logic combining password hashing and time-based OTP validation. Advanced production features (e.g., rate limiting or account lockout policies) are optional.

Note: The server should refuse any file transfers until both factors are successfully verified.

Day 3: Steganographic Extraction (Lab 8)

Task: Recover the "Architect's Manifesto" from evidence.png.

Requirement: The message is hidden using LSB steganography via the stepic library.

Instructions:

- Open the image using Pillow (PIL)
- Decode the message using stepic.decode()
- The message begins with: **"The vault is..."**
- Save the recovered text to architect_manifesto.txt

Implementation Note: The stepic library returns data in bytes; convert it to a readable string using .decode('utf-8'). If you choose to implement manual extraction, consider using the three RGB channels and look for the signature **"The Architect"** to identify the end of the message.

The output of your steganography script (i.e., the recovered text file) serves as the **payload** for the next stage of the pipeline.

Day 4: Cryptography & Secure Socket Transfer (Labs 0, 1, 9)

Task: Encrypt the manifesto and transmit it to the server you scanned on Day 1.

Requirement:

- **Encryption:** Protect architect_manifesto.txt using cryptography.fernet.
- **Transport:** Establish a socket-based connection.
- **The Handshake:** The Client must provide the Password and the current OTP (from Day 2).
- **Delivery:** Only after the server validates the MFA should the Client send the encrypted bytes.

Note: The payload is transformed into an encrypted format (bytes) to ensure it cannot be read if intercepted. Fernet tokens are already base64-encoded bytes; ensure your socket handles byte-stream transmission correctly.

Day 5: Validation & Project Reporting

Task: Document your findings + system architecture and prove functionality.

Deliverables:

- **Codebase:** ZIP file of all .py scripts
- **Demo Video (3 minutes):**
  - *Show Secret extraction from evidence.png.*
  - *Show Client-side authentication and encryption.*
  - *Show Server-side receipt and successful decryption to plain text.*

  Final Report: 3–4 pages (two-column format). The report should clearly explain your system design and implementation, including how your Day 1 network scan informed your Day 4 connection setup, as well as how the MFA logic was integrated. You may structure your report using the following sections:

  - *I. Network Discovery Phase*
  - *II. Authentication Configuration Phase*
  - *III. Forensics Analysis Phase*
  - *III. Cryptography and Secure Transport Phase*
  - *IV. System Integration Challenges*

Alternative Submission Option:
You may upload your codebase and demo video to a GitHub repository (e.g., within a final exam folder) and include the repository link in your Final Report document instead of submitting them separately.

Grading Rubric (100 Points)

- **Steganography Decoding accuracy (20 pts)** – Full manifesto recovered.
- **MFA Implementation (25 pts)** – Password + OTP functionality
- **Cryptographic Security (20 pts)** – Correct use of Fernet for encryption/decryption
- **Socket Reliability (20 pts)** – Stable communication and evidence of the Day 1 "Pre-check."
- **Professional Documentation and Demo (15 pts)** – Report and demo quality

---

Best of Luck!